
Hierarchical and Graph-Based Point Cloud Segmentation: A Comparative Study on ShapeNet Part Segmentation

Aishvarya Srivastava
College of Engineering
Carnegie Mellon University
aishvars@andrew.cmu.edu

Nandini Chanda
College of Engineering
Carnegie Mellon University
nchanda@andrew.cmu.edu

Serena Gomez
College of Engineering
Carnegie Mellon University
serenag@andrew.cmu.edu

Abstract

We study the problem of 3D part segmentation on the ShapeNet benchmark, where the goal is to assign a semantic part label to each point in a point cloud of a 3D object. We compare three deep learning architectures: PointNet serves as the baseline, PointNet++ as a hierarchical variant, and DGCNN as a dynamic graph-based variant. PointNet++ achieves the highest test-set instance-averaged mIoU of 82.27%, outperforming the PointNet baseline (79.79%) and DGCNN (80.44%). A robustness analysis reveals a striking asymmetry: PointNet’s global pooling holds up well under point cloud subsampling, retaining 99% of its mIoU at 128 points, while PointNet++ collapses to 48% of full performance at the same density, exposing a fundamental tradeoff between local-structure sensitivity and deployment robustness.

GitHub Repo Link: <https://github.com/aishvars/ShapeNet>.

1 Introduction

Understanding the fine-grained structure of 3D objects is a foundational problem in computer vision, robotics, and computer-aided design. Part segmentation, which is the task of labeling each point in a 3D shape with its constituent semantic part (e.g., the wing, fuselage, and engine of an airplane), is a core capability for applications such as automated assembly planning, robotic grasping, quality inspection, and 3D shape retrieval. Unlike 2D image segmentation, 3D point cloud segmentation must contend with data that is inherently unordered, irregularly sampled, and lacks the regular grid structure that convolutional neural networks exploit so effectively.

The ShapeNet Part dataset [Yi et al., 2016] provides 16,880 3D CAD models across 16 object categories annotated with 50 part labels, and has become the standard benchmark for evaluating point cloud part segmentation methods. Each shape is represented as a point cloud of 2,048 points with coordinates and surface normals. A successful model must learn to be invariant to the ordering of input points while remaining sensitive to local geometric structure.

If models can reliably segment parts at test time, new capabilities become possible: a robotic arm could identify the handle of a mug before grasping it; a quality-control system could detect malformed components in a manufactured part; and shape retrieval systems could find objects that share a specific sub-component even if the overall shape differs. In this work, we compare three architectures that take fundamentally different approaches to encoding 3D point cloud structure and analyze not only their accuracy but also their robustness and efficiency tradeoffs.

2 Methods

Baseline: PointNet

We use PointNet [Qi et al., 2017a] as our baseline. PointNet processes each point independently through a series of shared multi-layer perceptrons (MLPs implemented as 1D convolutions), then aggregates all point features into a single global descriptor via a symmetric max-pooling operation. The global descriptor is concatenated with per-point local features and a one-hot category vector before a final segmentation MLP predicts part labels. To handle the geometric transformation ambiguity inherent in point clouds, PointNet learns input and feature transformation matrices (T-Nets) that align the point cloud to a canonical orientation before processing; these are regularized by an orthogonality loss on the feature transform matrix.

PointNet is a natural baseline for this task because it is explicitly designed for unordered point sets and requires no voxelization or projection, preserving the raw 3D structure. Its key limitation is that the global max-pool discards all spatial relationships between points - each point’s contribution to the global descriptor is independent of its neighbors. This design makes PointNet fast and robust, but leaves local geometric structure entirely unmodeled.

Variant 1: PointNet++

PointNet++ [Qi et al., 2017b] extends PointNet by introducing a hierarchical encoder that explicitly captures local geometric context at multiple spatial scales. The encoder uses three stages of Set Abstraction (SA), each of which: (1) samples a set of centroids via farthest-point sampling (FPS), (2) groups nearby points within a fixed radius using ball-query, and (3) applies a mini-PointNet to each group to extract local features. This produces a progressively coarser but more geometrically informed representation. For segmentation, a decoder based on Feature Propagation (FP) interpolates features back to the original point resolution using inverse-distance-weighted interpolation and skip connections from the encoder, mirroring the U-Net structure.

We hypothesize that PointNet++ will outperform PointNet on this task because part segmentation requires distinguishing geometrically similar regions based on local context. For example, the back of a chair and the seat are spatially proximate; correctly labeling them requires reasoning about the local shape neighborhood, which PointNet’s global pooling cannot capture.

Variant 2: DGCNN

DGCNN [Wang et al., 2019] takes a fundamentally different approach by constructing a dynamic k -nearest-neighbor graph in *feature space* at each layer, rather than in fixed Euclidean space. Each layer computes EdgeConv operations: for each point i and each of its k nearest neighbors j in the current feature embedding, it processes the concatenation $[f_i \parallel (f_j - f_i)]$ through a shared MLP, then max-pools over neighbors to produce the updated feature for point i . Crucially, the graph is recomputed from scratch at every layer using the evolving feature representations, so semantic similarity - not just geometric proximity - drives the neighborhood structure in deeper layers. Features from all four EdgeConv layers are concatenated per-point and aggregated into a global descriptor, which is then combined with the per-point features and the category one-hot vector before the segmentation head.

We hypothesize that DGCNN’s dynamic graph structure will allow it to capture long-range semantic relationships that PointNet++ misses due to its fixed Euclidean radius grouping. However, computing pairwise distances at each layer over all N points incurs $O(N^2)$ memory, which may limit performance relative to the hierarchical approach at the same compute budget.

Training Details

All models are trained for 50 epochs on all 16 ShapeNet categories using the Adam optimizer with an initial learning rate of 10^{-3} , cosine annealing to 10^{-5} , weight decay of 10^{-4} , and cross-entropy loss with label smoothing of 0.2. Input point clouds use xyz coordinates concatenated with surface normals (6-dimensional input). Training data is augmented with random rotation around the vertical axis and coordinate jitter ($\sigma = 0.01$). PointNet additionally applies the T-Net orthogonality regularizer with weight 10^{-3} . All experiments are run on a single NVIDIA Tesla T4 GPU.

3 Results and Discussion

Core Evaluation

Table 1: Test-set performance on ShapeNet Part Segmentation (all 16 categories, 2,874 test shapes). Instance mIoU is the primary metric; category mIoU averages per-category means; point accuracy is per-point classification accuracy.

Model	Inst. mIoU (%)	Cat. mIoU (%)	Point Acc. (%)
PointNet (baseline) [Qi et al., 2017a]	79.79	73.47	91.52
PointNet++ (variant 1) [Qi et al., 2017b]	82.27	75.80	92.84
DGCNN (variant 2) [Wang et al., 2019]	80.44	73.49	92.30

We evaluate all models on the standard ShapeNet Part Segmentation test set using instance-averaged mean Intersection over Union (mIoU) as the primary metric. Instance mIoU averages the per-shape mIoU across all test shapes, where each shape’s mIoU is computed only over the part labels belonging to its category. This metric is preferred over point accuracy because part sizes are highly imbalanced: a metric dominated by large parts (e.g., the seat of a chair) would mask errors on small but semantically important parts (e.g., armrests).

Table 1 shows the main results. PointNet++ achieves the highest instance mIoU of 82.27%, a 2.48 percentage point improvement over the PointNet baseline (79.79%), confirming our hypothesis that hierarchical local feature learning benefits part segmentation. DGCNN reaches 80.44%, improving over the baseline but falling short of PointNet++. This is consistent with our hypothesis about the $O(N^2)$ computational overhead of dynamic graph construction limiting effective training within our compute budget relative to the ball-query approach of PointNet++.

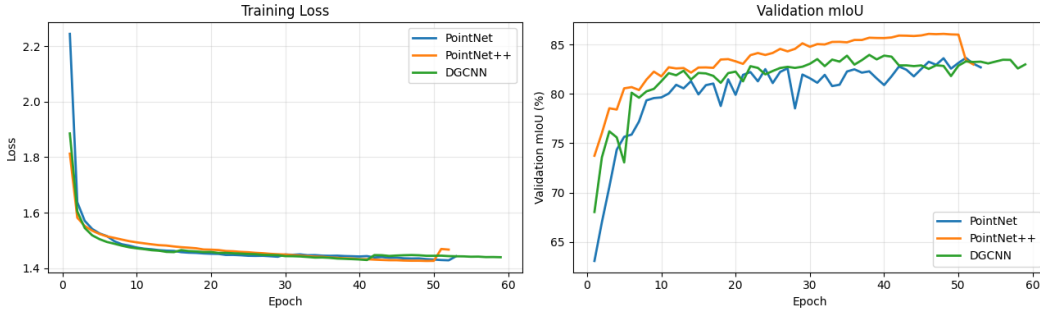


Figure 1: Training loss (left) and validation instance mIoU (right) over 50 epochs. PointNet++ achieves the highest peak validation mIoU of 86.07% at epoch 48. All three models converge stably, with the cosine learning rate schedule producing smooth final-epoch behavior.

Figure 1 shows the training dynamics. All three models converge stably, with training loss decreasing monotonically and validation mIoU generally increasing across epochs. Notably, the validation mIoU numbers are higher than test mIoU for all models (PointNet++ peaks at 86.07% on validation vs. 82.27% on test), suggesting mild overfitting to the validation distribution and indicating that further regularization or longer training may help.

Analysis 1: Per-Category mIoU Breakdown

Figure 2 shows per-category mIoU across all 16 object classes. PointNet++ leads on the majority of categories, with the largest advantages on geometrically complex shapes with many parts such as Motorbike (6 parts) and Lamp (4 parts), where local neighborhood context is most informative. All three models perform well on geometrically simple categories with few, large parts - Guitar (3 parts), Knife (2 parts), and Laptop (2 parts) - where even the global pooling of PointNet suffices to separate parts cleanly. The most difficult category for all models is Rocket, which has fine-grained elongated structure and constitutes one of the smallest categories in the dataset. Interestingly, DGCNN and PointNet perform comparably across most categories, with DGCNN’s dynamic graph

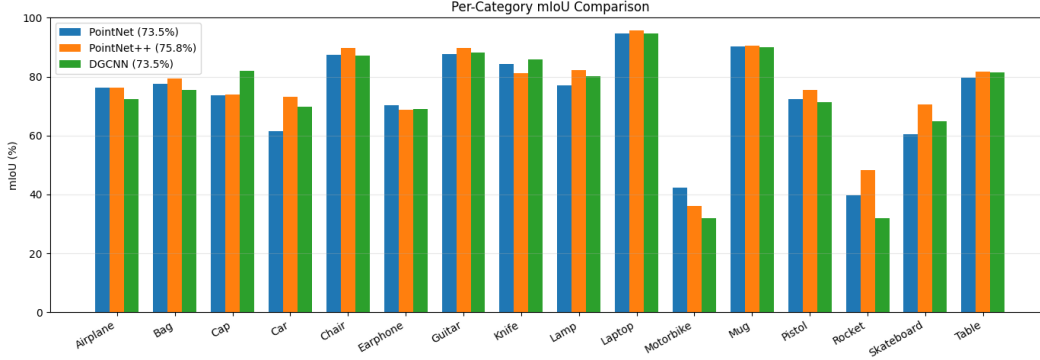


Figure 2: Per-category mIoU for all three models across 16 ShapeNet object categories. PointNet++ leads on most categories, with the largest gains on geometrically complex objects such as Motorbike and Lamp. All three models struggle on Rocket, which has fine-grained, elongated structure and the fewest training examples.

construction providing minimal advantage over PointNet’s global pooling on the simpler shape categories, suggesting the benefit of dynamic graph structure is concentrated in geometrically complex objects.

Analysis 2: Robustness to Test-Time Point Cloud Density

Table 2: Instance mIoU (%) as a function of test-time point cloud density. All models are trained on 2,048 points and evaluated on randomly subsampled clouds. Bold indicates best model at each density.

Model	2048 pts	1024 pts	512 pts	256 pts	128 pts
PointNet	79.94	80.14	80.05	79.74	79.02
PointNet++	82.32	81.78	75.61	60.86	48.28
DGCNN	80.46	80.60	77.04	69.10	62.40

A key question for practical deployment is how models behave when the test-time point cloud density differs from training. Lidar-based robotics systems, for instance, may produce sparser observations at greater distances. We evaluate all models by randomly subsampling the test point clouds to 1024, 512, 256, and 128 points without any retraining.

Table 2 and Figure 3 reveal a striking asymmetry. PointNet is nearly *perfectly* robust to density variation: its mIoU drops only from 79.94% at full density to 79.02% at 128 points - a relative loss of only 1.2%. In contrast, PointNet++ degrades catastrophically, collapsing from 82.32% to 48.28% (a 41.3% relative drop). DGCNN occupies the middle ground, dropping from 80.46% to 62.40% (a 22.4% relative drop).

This result directly follows from each architecture’s inductive biases. PointNet’s global max-pooling is by construction invariant to the number and ordering of input points - the most informative points still survive the max operation regardless of density. PointNet++’s ball-query grouping, by contrast, relies on fixed Euclidean radii tuned for 2,048-point density; at lower densities, many query balls become nearly empty, degrading the quality of local feature estimation. DGCNN’s k -NN graph degrades more gracefully because it always finds exactly k neighbors in feature space regardless of density, though accuracy still declines as the point set becomes too sparse to capture fine geometric detail.

This finding has direct implications for model selection: in deployment environments where point cloud density may vary (outdoor robotics, commodity depth sensors), PointNet’s architectural simplicity translates to a robustness advantage that its lower peak accuracy understates.

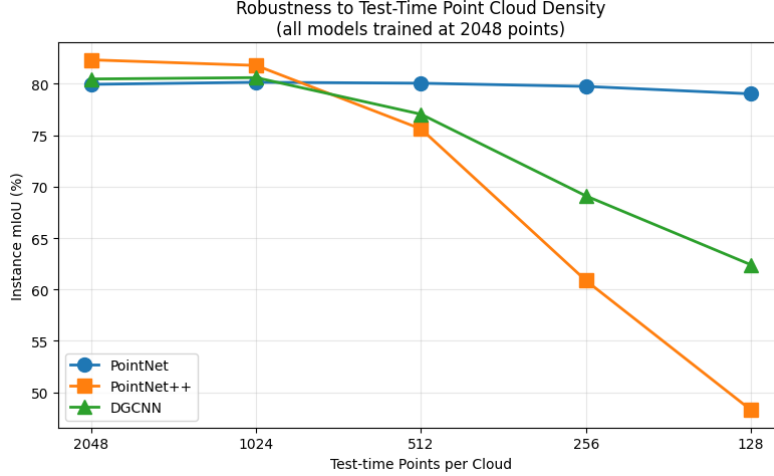


Figure 3: Instance mIoU as a function of test-time point cloud density (log scale). All models are trained on 2,048 points. PointNet (blue) degrades almost imperceptibly across the full range, while PointNet++ (orange) collapses sharply below 512 points. DGCNN (green) occupies the middle ground.

4 Conclusion

We compared PointNet, PointNet++, and DGCNN on the ShapeNet Part Segmentation benchmark. PointNet++ achieves the highest test-set instance mIoU (82.27%), confirming that hierarchical local feature learning benefits part segmentation by capturing the geometric neighborhood context that PointNet’s global pooling discards. DGCNN’s dynamic graph convolution provides a smaller improvement over the baseline (80.44%), likely because the $O(N^2)$ graph construction limits effective scaling within our compute budget.

The most surprising finding is the density robustness analysis. Despite having the *lowest* peak accuracy among the three, PointNet is dramatically more robust to point cloud subsampling, retaining 99% of its mIoU at 128 points while PointNet++ retains only 59%. This reveals that the hierarchical local structure that drives PointNet++’s accuracy advantage is also its Achilles heel under distribution shift.

Given more time and compute, we would explore three directions. First, training all models for 100 epochs (vs. 50 here) would likely reduce the validation-test mIoU gap observed in all three models. Second, evaluating PointNet++ with a multi-scale grouping (MSG) strategy, which uses multiple radii at each SA layer - may recover some density robustness while preserving accuracy. Third, training a Point Transformer [Zhao et al., 2021] would determine whether self-attention over point features closes the gap with PointNet++’s accuracy while improving robustness.

Collaboration Statement

Aishvarya implemented the PointNet baseline, the training pipeline, and the per-category mIoU analysis (Contribution 3). **Nandini** implemented PointNet++, including farthest-point sampling, ball-query grouping, set abstraction, and feature propagation (Contribution 1). **Serena** implemented DGCNN including the dynamic k -NN graph construction and EdgeConv layers, and designed the test-time density robustness experiment (Contributions 2 and 4). All members contributed equally to writing. Claude (Anthropic) was used to assist with initial code structure, LaTeX formatting, and proofreading.

References

- Li Yi, Vladimir G Kim, Duygu Ceylan, I-Chao Shen, Mengyan Yan, Hao Su, Cewu Lu, Qixing Huang, Alla Sheffer, and Leonidas Guibas. A scalable active framework for region annotation in 3d shape collections. In *ACM Transactions on Graphics (TOG)*, volume 35, pages 1–12. ACM, 2016.
- Charles R Qi, Hao Su, Kaichun Mo, and Leonidas J Guibas. Pointnet: Deep learning on point sets for 3d classification and segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 652–660, 2017a.
- Charles R Qi, Li Yi, Hao Su, and Leonidas J Guibas. Pointnet++: Deep hierarchical feature learning on point sets in a metric space. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017b.
- Yue Wang, Yongbin Sun, Ziwei Liu, Sanjay E Sarma, Michael M Bronstein, and Justin M Solomon. Dynamic graph cnn for learning on point clouds. *ACM Transactions on Graphics (TOG)*, 38(5): 1–12, 2019.
- Hengshuang Zhao, Li Jiang, Jiaya Jia, Philip HS Torr, and Vladlen Koltun. Point transformer. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 16259–16268, 2021.